# Table of Contents

1 Introduction....1
2 Top-down layered static view....3
3 Detailed design of the touch_buttons module....5
3.1 TouchButtons Class....6
3.1.1 Purpose....6
3.1.2 Design & Threading....6
3.1.3 API Reference....6
3.1.3.1 BUTTONS Constants....6
3.1.3.2 Methods:....6
3.1.4 Error handling....6
3.1.5 Attributes for module test....6
3.2 Module test....7

# 1 Introduction

The oradio_control is based on a state machine.

## 1.1 Introduction into UML state diagrams

A state diagram models the behavior of a system or an object in response to events. A state diagram mainly consists of states, events, guards, transitions, and actions. It's a behavioral diagram and it represents the behavior using finite state transitions.

### 1.1.1 `Initial state`

It shows the starting point of the system. It's represented as a filled circle

### 1.1.2 `Final State`

It shows the ending point of the system. It's represented as a circle with a dot inside

### 1.1.3 `Entry Point`

An input connector for control flow into an activity from outside (named) — used when connecting activities across activity partitions or when reusing activity nodes.

### 1.1.4 `Exit Point`

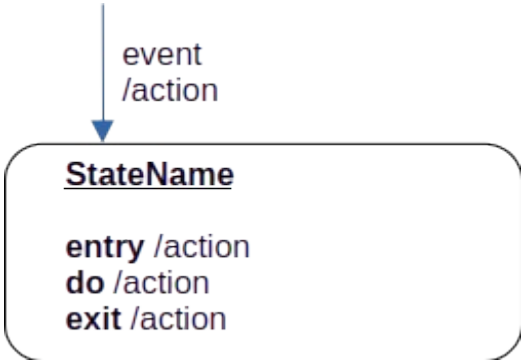
An output connector for control flow leaving an activity to outside; used to indicate exit from an activity (named).

### 1.1.5 Events

An event can be a signal,trigger or a change in the environment of the system. Events may or may not lead to a change in the system's state

### 1.1.6 States

A state represents the conditions or circumstances of an object of a class at an instant of time.




- **StateName**: The name of the state
- **event:** trigger that causes the transition.
- **/action**: An operation executed in response to an accepted event

State actions are invoked when the system is in a specific state. There are 3 types of state actions:

- **Entry** actions: they are executed when the system enters the state
- **Do** actions: they are continuously executed while the system is in the state
- **Exit** actions: they are executed when the system exits the state

### 1.1.7 Transitions

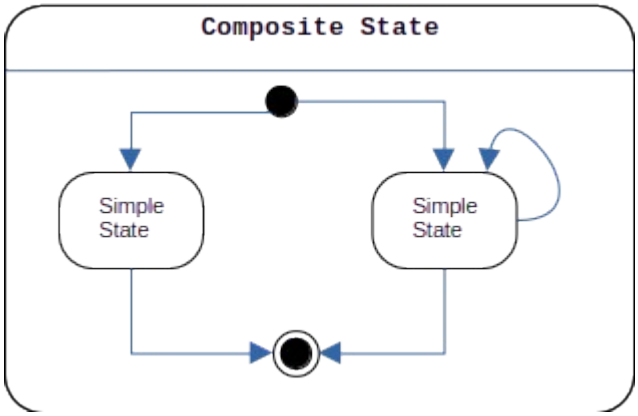
Transitions are shown as arrows from one state to another. They may have three indicators written on them:

- The event that triggers the transition
- The guard statement which is a condition that must be true for the transition to take place
- An action statement that will happen as a result of the transition

```
[isSpotifyConnected]
ButtonPlay
/playSound()
/setLed()
/startSpotifyPlayback()
```

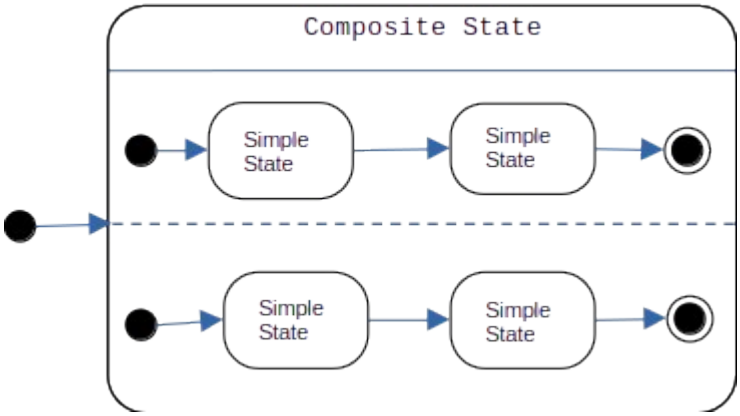
### 1.1.8 Composite State

A state that has substructure and internal behavior. It can contain one or more regions, which can contain one or more states. The state diagrams are executed in parallel.



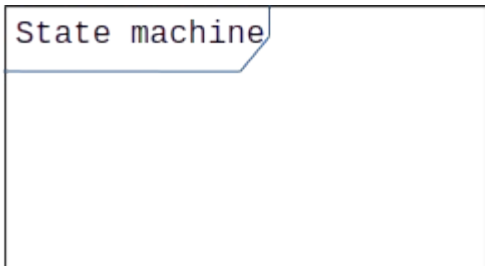

### 1.1.9 Concurrent regions

Represent separate areas of concern within a state, allowing for independent state transitions in each region, simultaneously.




### 1.1.10 Package

The UML design symbol for a package is depicted as a tabbed folder, which represents a container for grouping related UML elements.



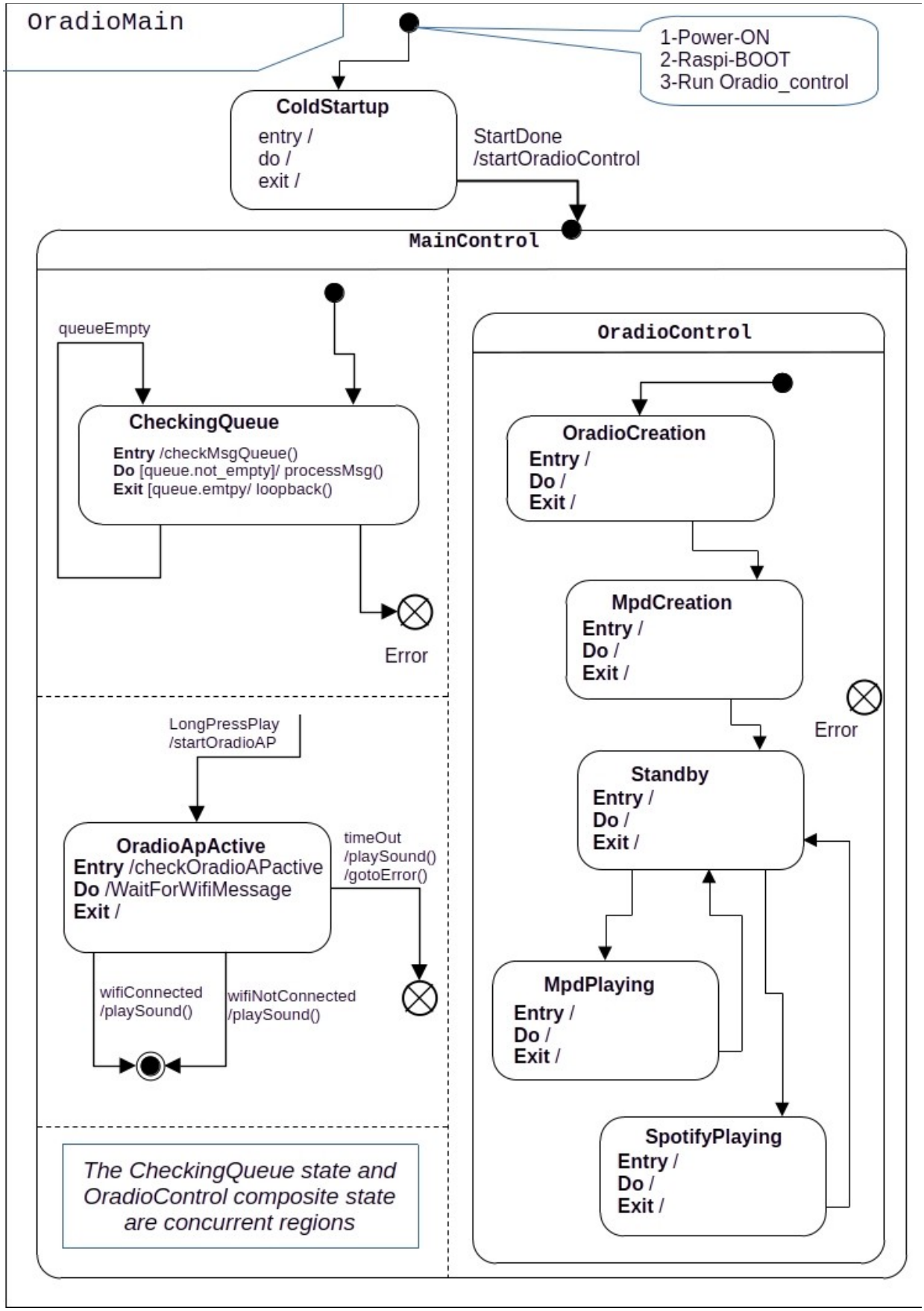
## 1.2 Introduction into State transition diagrams

a state-transition table is a table showing what state a finit-state machine will move to, based on the current state and other inputs. It is essentially a truth table in which the inputs include the current state along with other inputs, and the outputs include the next state along with other outputs.

| Current State | Event | Condition(guards) | Next State | Action |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

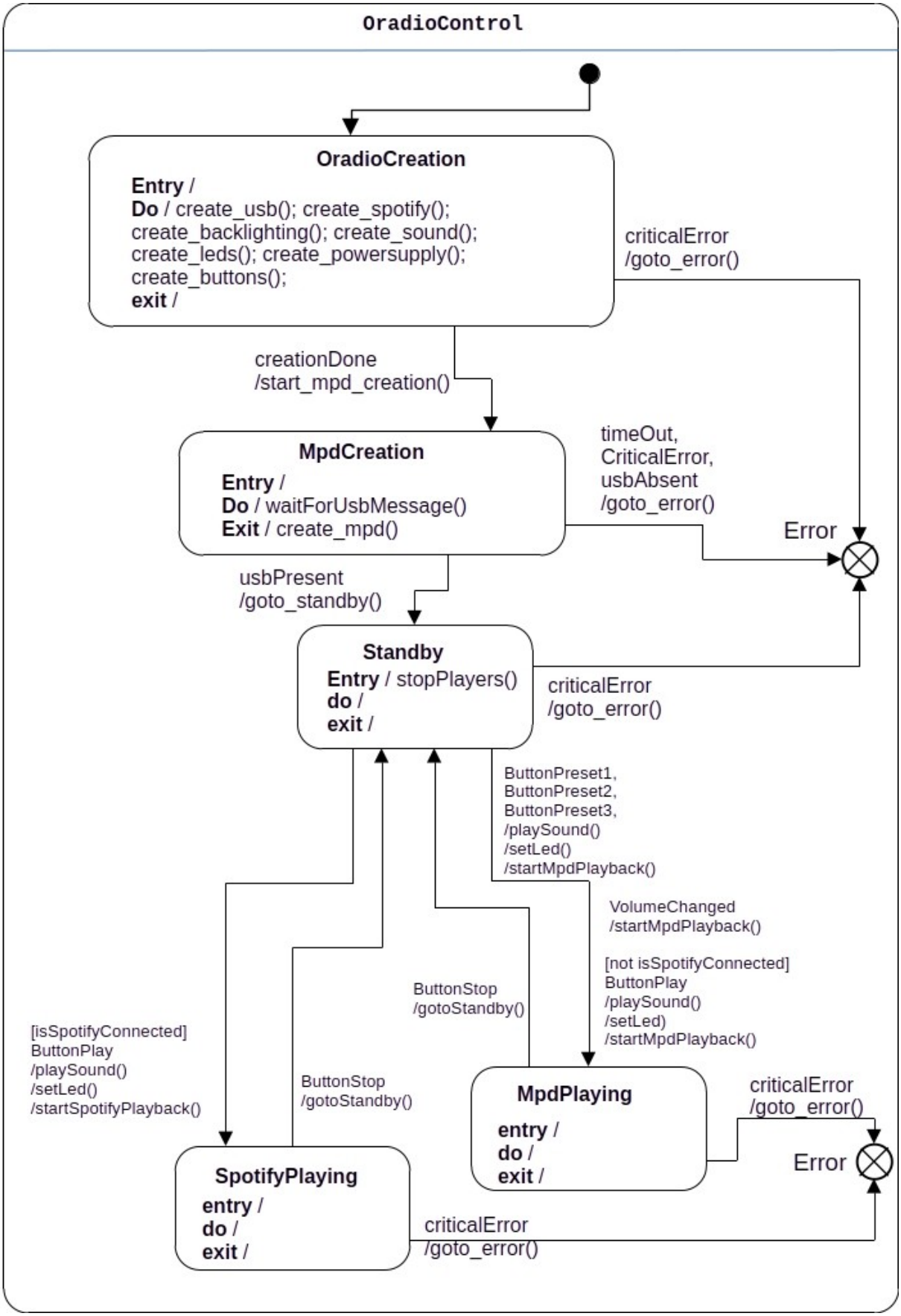
# 2 The Oradio state diagrams views

## 2.1 The OradioMain state diagram



## 2.2 The OradioMain composite state

### 2.2.1 State diagram



### 2.2.2 State table

| Current State | Event | Condition | Next State | Action |
|---|---|---|---|---|
| OradioCreation | criticalError | None | ErrorHandling | goto_error() |
| | creationDone | None | MpdCreation | start_mpd_creation() |
| MpdCreation | usbAbsent | None | ErrorHandling | goto_error() |
| | usbPresent | None | ErrorHandling | goto_error() |
| | timeOut | None | ErrorHandling | goto_error() |
| | mpdCreationDone | None | Standby | |
| | criticalError | None | ErrorHandling | goto_error() |
| Standby | ButtonPreset1, ButtonPreset2, ButtonPreset3 | None | MpdPlaying | playSound() setLed() startMpdPlayback() |
| | VolumeChanged | None | MpdPlaying | startMpdPlayback() |
| | ButtonPlay | not isSpotifyConnected | MpdPlaying | playSound() setLed) startMpdPlayback() |
| | VolumeChanged | None | MpdPlaying | startMpdPlayback() |
| | CriticalError | None | ErrorHandling | goto_error() |
| | ButtonPlay | isSpotifyConnected | SpotifyPlaying | playSound() setLed() startSpotifyPlayback() |
| MpdPlaying | ButtonStop | None | Standby | gotoStandby() |
| | CriticalError | None | ErrorHandling | goto_error() |
| SpotifyPlaying | ButtonStop | None | Standby | gotoStandby() |
| | CriticalError | None | ErrorHandling | goto_error() |

# 3 Top-down layered static view

TBD

# 4 Detailed design of the oradio_control module.

## 4.1 Classess

### 4.1.1 Purpose

### 4.1.2 Design & Threading

### 4.1.3 API Reference

#### 4.1.3.1 BUTTONS Constants

#### 4.1.3.2 Methods:

### 4.1.4 Error handling

### 4.1.5 Attributes for module test

## 4.2 Module test